Lab 5: Geometric data decomposition implementation and analysis

Parallelism

Universitat Politècnica de Catalunya (UPC)
Facultat d'Informàtica de Barcelona (FIB)

Authors:

Ferran Mesas
ferran.mesas@estudiantat.upc.edu

Ishak Felfoul
ishak.felfoul@estudiantat.upc.edu

June 8, 2026

Table of Contents

1. Introduction

This report *aims* to be as short as possible without losing technical details. The writers hope it will be to the reader's liking.

To avoid repetition, some common elements used in the following sections are presented here, thus enabling their omission later.

Common code changes

As in the previous report, some changes to the codes are common across all the implementations, these are the following:

- All the code inside the procedure `mandel_simple` is wrapped inside of a `#pragma omp parallel` region, with its corresponding braces giving it all the scope of the procedure.
- In the beginning, these two values are always retrieved:

```
int thread_id = omp_get_thread_num();
int num_threads = omp_get_num_threads();
```

- Adding a `#pragma omp atomic` above of the histogram increment `histogram[M[y][x] - 1]++`, since the memory position being incremented depends on a runtime value and several threads could otherwise increment it simultaneously, causing a *data race*.
- Adding a `#pragma omp critical(display_output)` region to the display logic containing the statements `XSetForeground(display, gc, color);` and `XDrawPoint(display, win, gc, x, y);`. The purpose of the label `display_output` is not to block non-related regions once a thread accesses this region, in this program's case, it could be omitted since there is no other critical region. The reason is that the first statement writes an internal variable of the library, afterwards read by the second one, without ensuring that this value does not change between these two operations, another thread could read from or write to it, causing a *data race*.

Additionally, it is important to mention that the correctness has been verified by comparing the image and the histogram of the sequential version to the one of each parallel implementation as the statement explains. These checks have been successfully passed.

2. 1D block geometric data decomposition by columns

2.1. Code changes

Source code: `mandel-omp-iter-block.cpp`

These first variable definitions calculate the first and last columns processed by this thread, assuming that all threads receive the same exact amount of columns. It also calculates how many columns would not be assigned to any thread.

```
int columns_per_thread = COLS/num_threads;
int remaining_columns = COLS%num_threads;
int thread_column_start = thread_id*columns_per_thread;
int thread_column_end = (thread_id + 1)*columns_per_thread;
```

The following code snippet is responsible for assigning the remaining columns to the threads, with a maximum imbalance of one column and keeping the same-thread columns consecutive.

```
// Assign an extra column to the remaining_columns initial threads.
if (remaining_columns != 0) {
    if (thread_id < remaining_columns) {
        // This is from the first ones, we assign
        // an extra element accounting for the offset
        // of the elements assigned to prior threads.
        thread_column_start += thread_id;
        thread_column_end += thread_id + 1;
    } else {
        // This is from the last ones, at this point,
        // we already assigned the remaining elments
        // to prior threads, we only need to account
        // for offsets arising from such assignments.
        thread_column_start += remaining_columns;
        thread_column_end += remaining_columns;
    }
}
```

Then, only the loop iterating over columns changes.

```
for (int x = thread_column_start; x < thread_column_end; x++)
```

2.2. Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.24	2.60	2.21	1.44	1.01	0.75	0.63	0.57	0.50	0.43
Speedup	1.00	1.24	1.47	2.25	3.19	4.30	5.16	5.71	6.49	7.47
Efficiency	1.00	0.62	0.37	0.28	0.27	0.27	0.26	0.24	0.23	0.23

Table 1: Overview of whole program execution metrics for 1D Block geometric data decomposition by columns.

Here it can be seen how the elapsed time does decrease with increasing number of threads, as one would expect, but when looking at the speedup it does not keep up with the increasing number of threads, it grows clearly slower. That directly impacts efficiency, which decreases rapidly until it remains constant at a low value. The cause of this will be shown below.

Overview of the Efficiency metrics in parallel fraction, ϕ=99.60%										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	62.29%	36.78%	28.29%	26.85%	27.28%	26.24%	24.27%	23.74%	24.03%
Parallelization strategy efficiency	100.00%	62.34%	36.83%	28.35%	26.98%	27.51%	26.67%	24.86%	24.55%	24.79%
Load balancing	100.00%	62.35%	36.84%	28.36%	27.00%	27.55%	26.73%	24.92%	24.64%	24.91%
In execution efficiency	100.00%	99.99%	99.96%	99.95%	99.93%	99.88%	99.79%	99.74%	99.66%	99.52%
Scalability for computation tasks	100.00%	99.91%	99.87%	99.79%	99.50%	99.15%	98.38%	97.63%	96.68%	96.91%
IPC scalability	100.00%	99.99%	99.98%	99.96%	99.74%	99.50%	98.86%	98.30%	97.52%	98.03%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	99.99%	99.99%	99.99%	99.99%	99.98%	99.98%
Frequency scalability	100.00%	99.92%	99.89%	99.83%	99.77%	99.66%	99.53%	99.32%	99.16%	98.88%

Table 2: Overview of the Efficiency metrics in parallel fraction for 1D Block geometric data decomposition by columns.

From this table, the conclusion to be drawn is that the inefficiency is caused almost entirely by a severe load imbalance, dropping slightly below 25% when reaching the

highest amounts of threads. This is due to the non-uniform workload in the input data, which will be discussed in greater depth in upcoming subsections. Other metrics related to the execution of the task itself indicate that it otherwise scales fairly well.

Overheads in executing implicit tasks										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3226416.07	1614641.69	807636.11	404163.33	270207.46	203375.33	163980.65	137701.45	119181.41	104036.35
Load balancing for implicit tasks	1.0	0.62	0.37	0.28	0.27	0.28	0.27	0.25	0.25	0.25
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	113.21	233.48	1793751.05	1404379.77	987572.48	728651.42	605562.09	545776.48	477973.6	414077.21

Table 3: Overheads in executing implicit tasks for 1D Block geometric data decomposition by columns.

Here the aforementioned load unbalancing appears directly once again, and it can also be appreciated in other ways: the *time in fork/join implicit tasks* is the average time each thread has to wait at barriers for other threads to finish, in this case, that time is very high precisely due to the load imbalance. Threads that finished wait for the rest in the implicit barrier of the parallel region. The severity is such that, from four threads onward, threads spend more time waiting for other threads than useful duration they actually have.

2.3. Paraver analysis

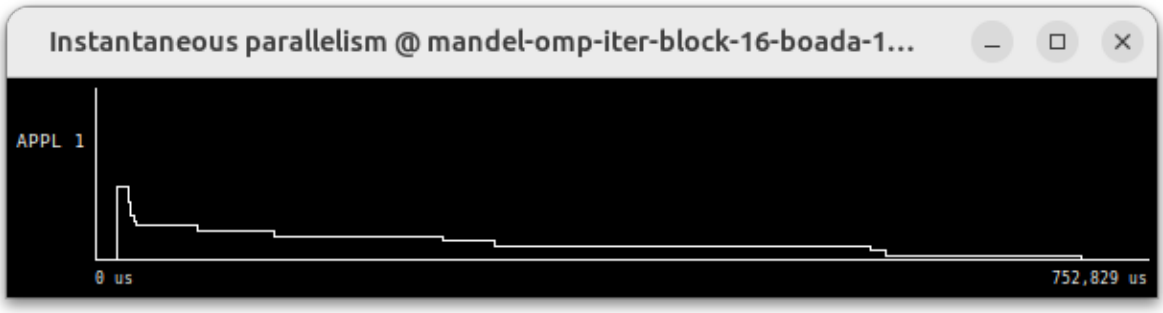

Figure 1: Instantaneous parallelism for 1D Block geometric data decomposition by columns.

Instantaneous parallelism shows how parallelism starts being initially excellent with that peak, though, it sharply declines as threads with the least load finish their task and they just wait for the rest in the implicit barrier.

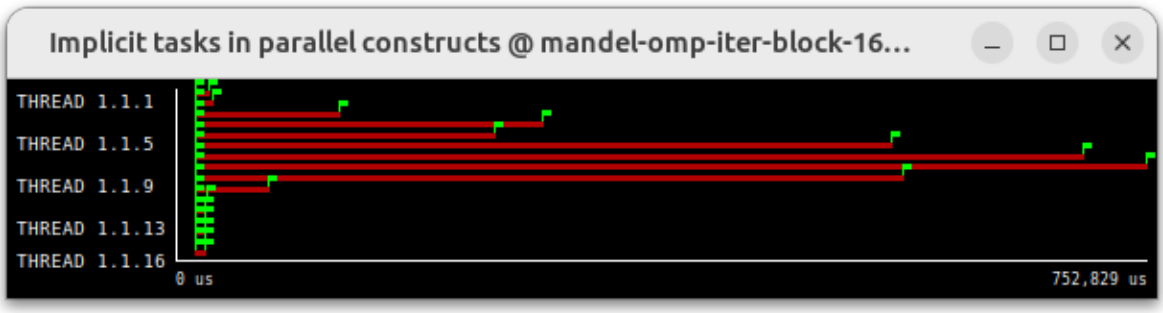

Figure 2: Implicit tasks in parallel constructs for 1D Block geometric data decomposition by columns.

The present figure further reinforces the visibility of the load imbalance: some threads whose tasks assigned likely contained columns where the numbers quickly escaped the Mandelbrot set finished very early, in contrast with the others that likely were assigned a column of the white blob or nearby, where numbers take longer to escape or reach the maximum number of iterations permitted, thus, assumed not to escape the set.

2.4. Memory analysis

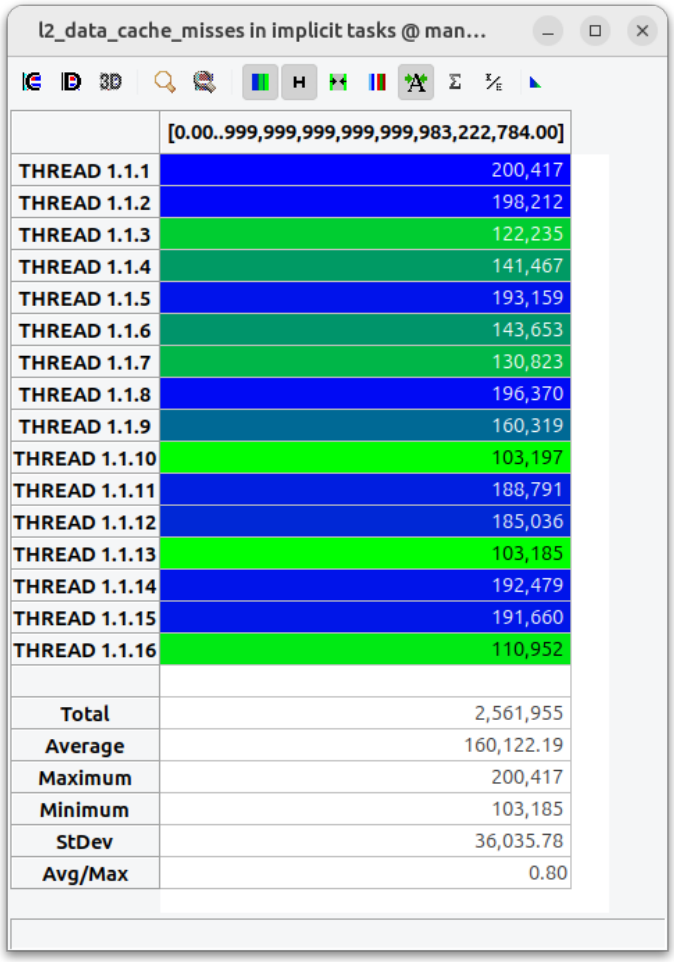

Figure 3: L2 data cache misses with 16 threads for 1D Block geometric data decomposition by columns.

Threads	Misses	Misses/Threads
1	1612336	1612336
2	1722634	861317
4	1897312	474328
8	2218417	277302
12	2640125	220010
16	2569570	160598
20	2841183	142059
24	2880057	120002
28	2990316	106797
32	1975623	61738

Table 4: L2 data cache misses for different numbers of threads for 1D Block geometric data decomposition by columns.

It can be seen how the number of total misses tends to increase as threads also increase. This is likely due to the division being made in a way in which almost by random chance causes the boundaries between the columns assigned to different threads to split cache lines. This causes false sharing, which increases the number of misses above the compulsory amount. The case of 32 threads is a bit special, because it does $512 \cdot 10/32 = 5120/32 = 160$, which taking into account that each cache line is 64 bytes and each `int` is 4 bytes, each block is exactly the size of 10 cache lines, hence, each boundary falls right in the middle of cache lines, which should prevent false sharing. The variability shown in the misses from different threads likely comes from a mix of chance about where the boundaries fell at and the chance of which thread allocated the cache line in its private cache first.

2.5. Strong scalability

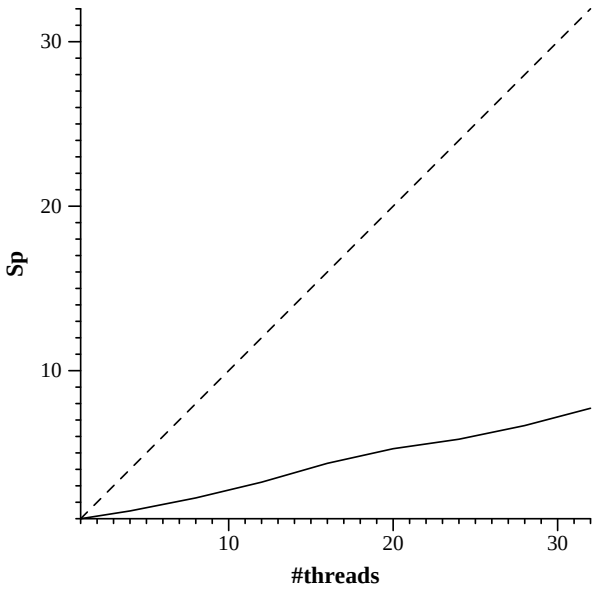

Figure 4: Speed-up with respect to sequential time for 1D Block geometric data decomposition by columns.

As it should be expected, the strong scalability is very bad, it follows a relatively but not completely straight line. The slope is approximately $1/4 = 0.25$, which implies that approximately four additional threads must be added to halve the time once, as compared to the ideal which would be only a single extra thread.

3. 1D block-cyclic geometric data decomposition by columns

3.1. Code changes

Source code: `mandel-omp-iter-block-cyclic.cpp`

Firstly, some utility macros are defined, cache lines are known to be of size 64 bytes and we use this size as the block size

```
#define CACHE_LINE_SIZE 64
#define BYTES_PER_BLOCK CACHE_LINE_SIZE
#define MIN(a, b) ((a) < (b) ? (a) : (b))
```

Then we have the amount of elements per cache line, used for each block.

```
int block_size = BYTES_PER_BLOCK/sizeof(int);
```

The loop iterating the columns is replaced by two loops, the outer loop determines which column blocks each thread processes based on the thread number, and the inner loop iterating such block, with a minimum in the guard to avoid out-of-bounds accesses in the last iteration of the outer loop in case the block size does not divide the row size.

```
for (int xx = thread_id*block_size;
     xx < COLS;
     xx += num_threads*block_size)
    for (int x = xx; x < MIN(xx + block_size, COLS); x++)
```

3.2. Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.24	1.63	0.83	0.43	0.29	0.23	0.19	0.16	0.14	0.14
Speedup	1.00	1.99	3.92	7.61	11.24	14.22	17.41	20.53	22.72	23.37
Efficiency	1.00	0.99	0.98	0.95	0.94	0.89	0.87	0.86	0.81	0.73

Table 5: Overview of whole program execution metrics for 1D Block-cyclic geometric data decomposition by columns.

This table shows how the speed-up for the maximum number of threads almost tripled as compared to the last implementation, which obviously also boosted efficiency by roughly the same coefficient. This signals that, at least partially, the load imbalance has been solved, as it will be shown below, since the most of the inefficiency was observed to be caused by that problem.

Overview of the Efficiency metrics in parallel fraction, ϕ=99.61%										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	99.84%	99.48%	98.41%	97.54%	93.82%	93.24%	92.78%	88.82%	80.29%
Parallelization strategy efficiency	100.00%	99.98%	99.75%	98.72%	98.01%	94.56%	94.34%	94.12%	90.38%	82.22%
Load balancing	100.00%	99.99%	99.79%	98.87%	98.28%	94.95%	94.99%	95.05%	91.55%	83.41%
In execution efficiency	100.00%	99.99%	99.96%	99.85%	99.74%	99.58%	99.32%	99.03%	98.73%	98.58%
Scalability for computation tasks	100.00%	99.86%	99.73%	99.69%	99.51%	99.22%	98.83%	98.58%	98.27%	97.65%
IPC scalability	100.00%	99.93%	99.85%	99.80%	99.70%	99.61%	99.33%	99.23%	99.15%	98.68%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	99.99%	99.99%	99.99%	99.99%	99.98%	99.98%
Frequency scalability	100.00%	99.94%	99.88%	99.89%	99.82%	99.62%	99.51%	99.36%	99.14%	98.97%

Table 6: Overview of the Efficiency metrics in parallel fraction for 1D Block-cyclic geometric data decomposition by columns.

As previously speculated, the load imbalance decreased massively, although it still persists somewhat, being especially noticeable when increasing from 24 threads to 32 threads. The

strategy and global efficiency are heavily tied to it, as it was already seen. The other metrics remain good and even perceived a very slight increase from the ones seen before.

Overheads in executing implicit tasks										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3231651.44	1618018.93	810098.86	405213.52	270628.32	203564.06	163492.69	136598.36	117444.89	103421.33
Load balancing for implicit tasks	1.0	1.0	1.0	0.99	0.98	0.95	0.95	0.95	0.92	0.83
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	114.84	198.84	2841.35	5404.17	1674.23	3117.66	3126.14	6063.82	6326.04	30641.7

Table 7: Overheads in executing implicit tasks for 1D Block-cyclic geometric data decomposition by columns.

Besides the explicit load imbalance already commented on, threads no longer spend most of the time waiting at the *fork/join*, ever, since the load imbalance has been partially fixed. The time waited there is now fairly reasonable.

3.3. Paraver analysis

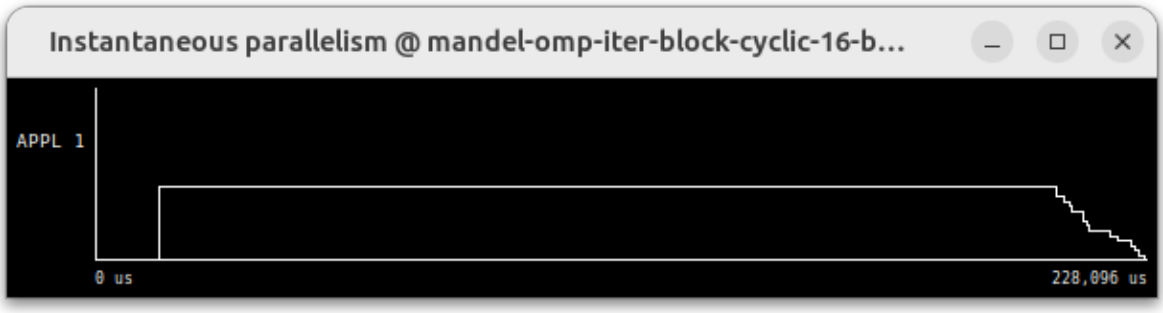

Figure 5: Instantaneous parallelism for 1D Block-cyclic geometric data decomposition by columns.

Instantaneous parallelism is excellent throughout all the execution, except in the end when it starts declining progressively as some tasks finish sooner, this is due to the mild load imbalance remaining.

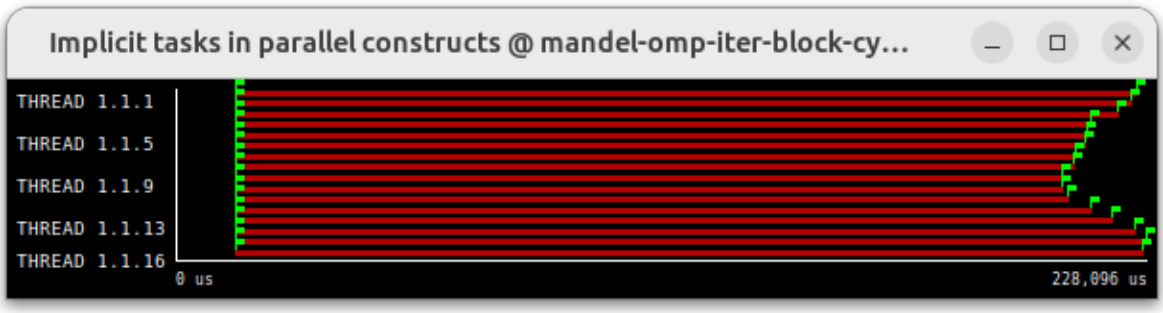

Figure 6: Implicit tasks in parallel constructs for 1D Block-cyclic geometric data decomposition by columns.

It can be effectively seen how the execution time of each task has become more balanced due to interleaving, which mitigates the load imbalance. That could be further improved by decreasing the size of the block to get more interleaving, but at the cost of the block size dropping below the size of a cache line, which would cause false sharing and decrease memory performance.

3.4. Memory analysis

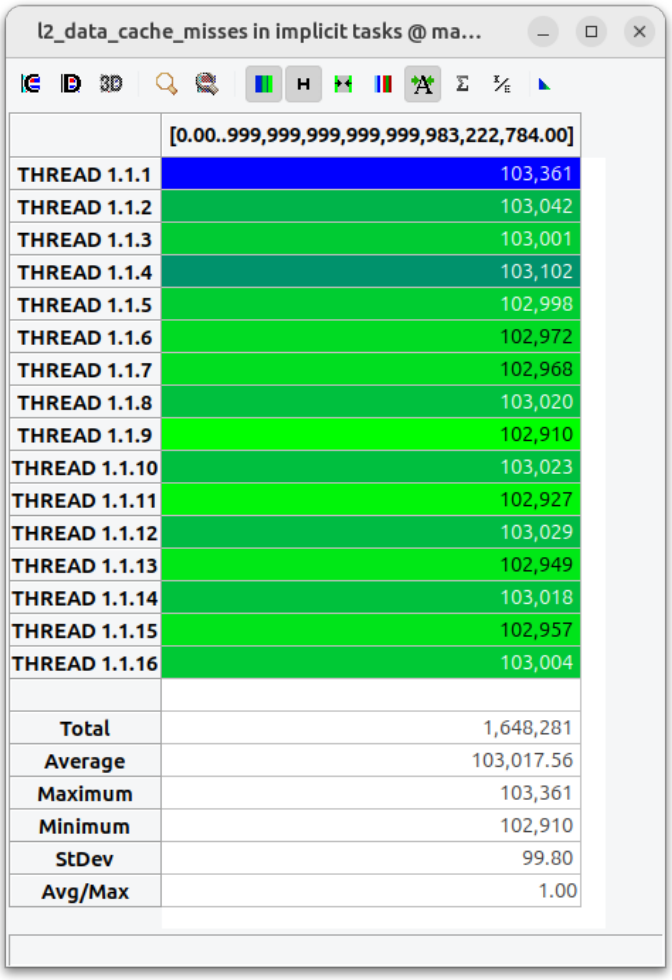

Figure 7: L2 data cache misses with 16 threads for 1D Block-cyclic geometric data decomposition by columns.

Threads	Misses	Misses/Threads
1	1611978	1611978
2	3175008	1587504
4	4701267	1175316
8	1649787	206223
12	1652154	137679
16	1654279	103392
20	1658654	82932
24	1660519	69188
28	1664495	59446
32	1664015	52000

Table 8: L2 data cache misses for different numbers of threads for 1D Block-cyclic geometric data decomposition by columns.

In principle, only compulsory misses should occur since, as mentioned earlier, we used the cache line size as block size, there should be no threads sharing cache lines and causing false sharing, on top of that, elements in a cache line are accessed consecutively, so there should not be evictions due to collision or capacity. Despite the prior reasoning, we see how from 1 thread to 4 threads misses increase, just to drop to the baseline at 8 threads and onward and remain stable. The most likely reason leading to this, is that most processors have `hardware prefetchers`, so when they detect accesses with a constant stride they may try to load nearby cache lines in advance or, in the case of the processor of one of the boada nodes, there seems to be a prefetcher that loads an extra cache line to L2 to complete the 128 bytes of a block when a line of cache is accessed in L2. What may be happening is that for small numbers of threads, the prefetcher may detect the stride and start allocating the prefetched lines in the private cache of the wrong core,

then when the core that actually has to write there requires its ownership, it has an extra miss there. Several block sizes such as two cache lines (128 bytes) have been tried, but they only reduce misses slightly and at the cost of increasing load imbalance and worsening speed-up overall, that is why a single cache-line block was chosen because it provides the best balance between load balance and memory performance. This seems to be a problem almost inherently tied to this access pattern. Regarding the variability in the misses for each thread, it can be seen that they have almost the same amount each, which fits with having almost only compulsory misses.

3.5. Strong scalability

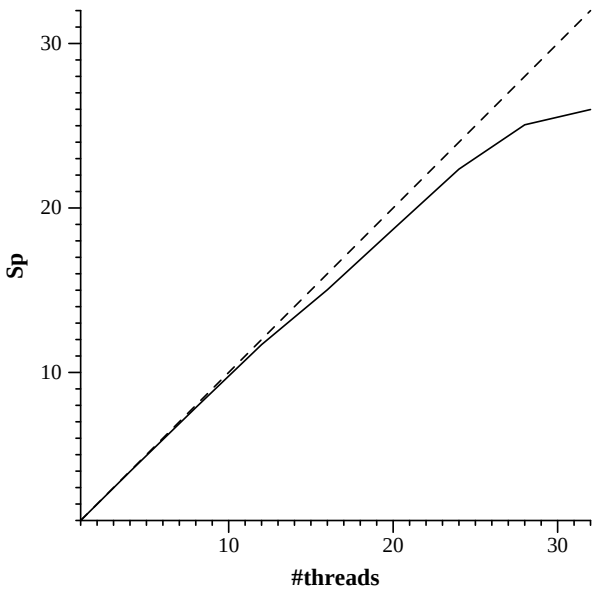

Figure 8: Speed-up with respect to sequential time for 1D Block-cyclic geometric data decomposition by columns.

For this implementation, the strong scalability is noticeably better than with the previous one. The speed-up stays very close to the ideal for up to 12 threads, starts declining more up to 24 threads and for 28 and 32 threads is where the decline is the most notorious. This implementation scales well, but after a certain point that scalability starts being compromised.

4. 1D cyclic geometric data decomposition by rows

4.1. Code changes

Source code: `mandel-omp-iter-cyclic.cpp`

This implementation is extremely simple, the only thing to do is to replace the outer loop with one that iterates interleaving rows based on the thread number.

```
for (int y = thread_id; y < ROWS; y += num_threads)
```

4.2. Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.22	1.62	0.82	0.42	0.28	0.22	0.18	0.15	0.13	0.12
Speedup	1.00	1.99	3.95	7.75	11.41	14.83	18.27	21.40	24.45	27.36
Efficiency	1.00	0.99	0.99	0.97	0.95	0.93	0.91	0.89	0.87	0.85

Table 9: Overheads in executing implicit tasks for 1D Cyclic geometric data decomposition by rows.

This table shows how speed-up keeps increasing almost perfectly up to 8 threads and, afterwards, a bit below that but still signaling a very good strong scaling. When looking at efficiency, it also improved noticeably as compared to the previous implementation, likely due to better load balance, as shown below.

Overview of the Efficiency metrics in parallel fraction, ϕ=99.59%										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	99.87%	99.86%	99.68%	99.26%	98.22%	98.27%	97.59%	96.73%	95.47%
Parallelization strategy efficiency	100.00%	99.94%	99.95%	99.78%	99.52%	98.82%	99.06%	98.84%	98.33%	97.72%
Load balancing	100.00%	99.96%	99.99%	99.93%	99.84%	99.32%	99.85%	99.73%	99.57%	99.45%
In execution efficiency	100.00%	99.98%	99.97%	99.86%	99.68%	99.50%	99.21%	99.11%	98.75%	98.25%
Scalability for computation tasks	100.00%	99.93%	99.91%	99.89%	99.74%	99.39%	99.21%	98.73%	98.38%	97.70%
IPC scalability	100.00%	100.00%	99.99%	99.98%	99.96%	99.72%	99.66%	99.35%	99.30%	98.78%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Frequency scalability	100.00%	99.93%	99.92%	99.91%	99.78%	99.67%	99.54%	99.38%	99.07%	98.91%

Table 10: Overview of the Efficiency metrics in parallel fraction for 1D Cyclic geometric data decomposition by rows.

It can be seen how load balancing remains nearly perfect even as the number of threads increase, this is a very obvious change one appreciates when looking at this table and comparing it to the previous implementations, where load imbalance was glaring or still problematic. Strategy and global efficiency have also improved as a result of that. Other metrics remain similar to the previous implementation.

Overheads in executing implicit tasks										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3209299.21	1605840.53	803075.15	401598.46	268126.19	201803.32	161750.67	135437.67	116510.78	102651.61
Load balancing for implicit tasks	1.0	1.0	1.0	1.0	1.0	0.99	1.0	1.0	1.0	0.99
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	122.24	1588.81	265.91	576.35	1773.02	2933.94	2188.23	2452.42	3625.92	4457.88

Table 11: Overheads in executing implicit tasks for 1D Cyclic geometric data decomposition by rows.

Load balancing in implicit tasks seems to have become perfect according to this table, except for the 32-thread case and not by much. Looking at the time spent at *fork/join*, it has been further reduced compared to the previous implementation, likely due to even better load balancing. Now the fraction of time spent waiting compared to the useful time is truly small.

4.3. Paraver analysis

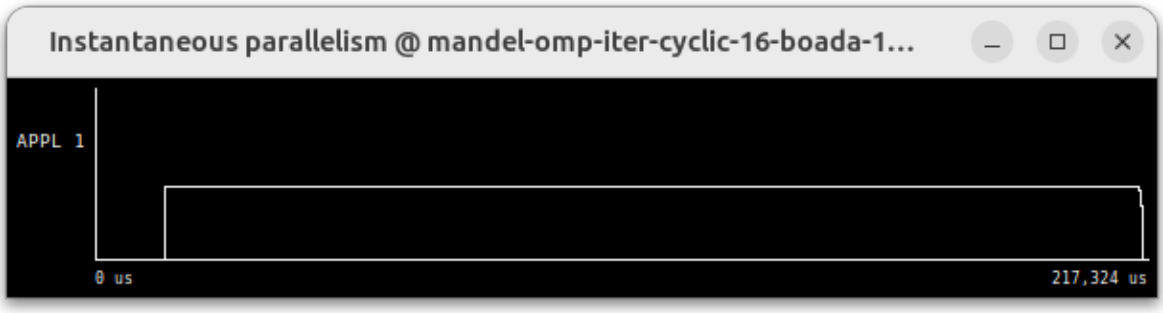

Figure 9: Instantaneous parallelism for 1D Cyclic geometric data decomposition by rows.

Instantaneous parallelism appears to be perfect throughout the entire execution, perhaps except for a very minimal part in the very end where it decreases almost in a vertical line, but not quite, before terminating. This is the best parallelism of all the implementations.

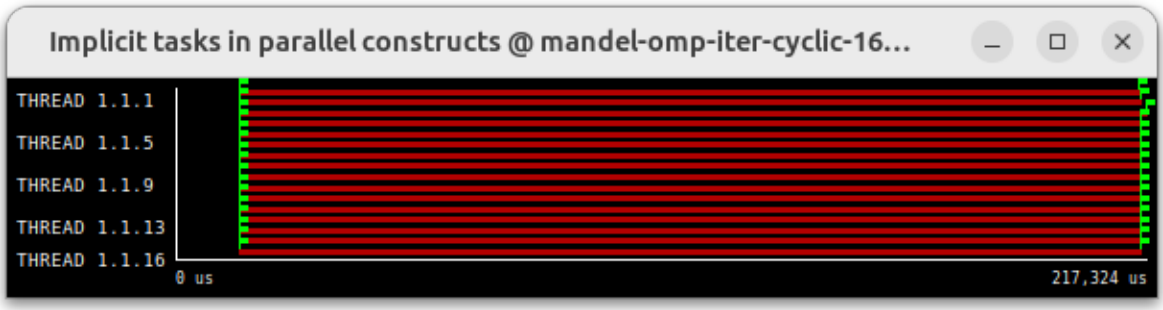

Figure 10: Implicit tasks in parallel constructs for 1D Cyclic geometric data decomposition by rows.

This figure further portrays the very high load balancing of this implementation: only a very tiny variation in one of the threads is visible at first sight. This is likely due to the increased interleaving that this pattern allowed to do, mitigating even more strongly the non-uniform workload of the input by avoiding assigning tasks too many elements spatially adjacent. This implementation allows threads to interleave work at 16 times finer granularity than the previous implementation without decreasing memory performance.

4.4. Memory analysis

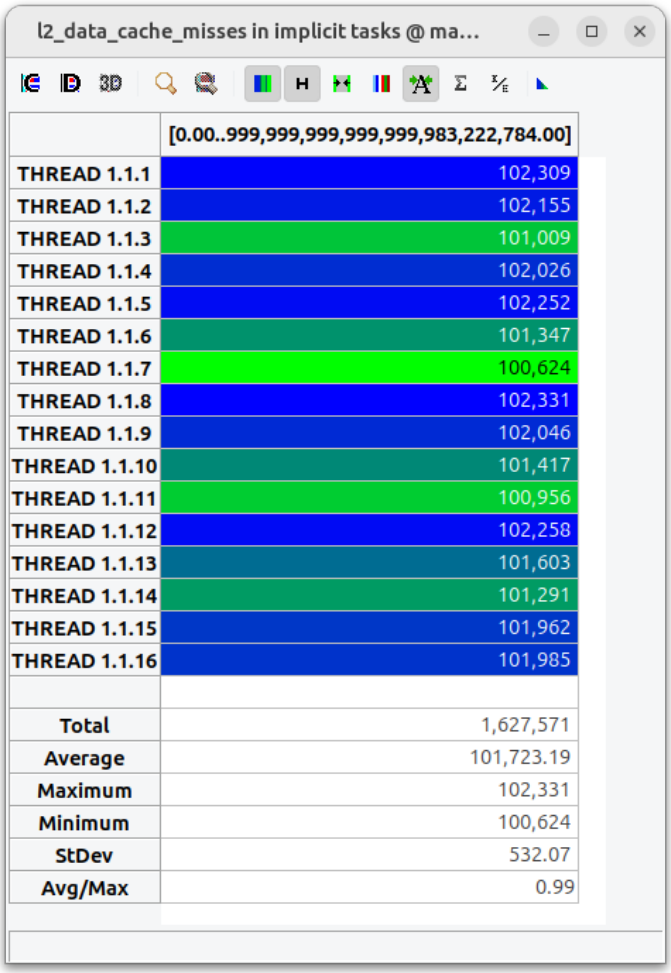

Figure 11: L2 data cache misses with 16 threads for 1D Cyclic geometric data decomposition by rows.

Threads	Misses	Misses/Threads
1	1611996	1611996
2	1617397	808698
4	1614054	403513
8	1616512	202064
12	1627056	135588
16	1634307	102144
20	1644412	82220
24	1647330	68638
28	1652704	59025
32	1649713	51553

Table 12: L2 data cache misses for different numbers of threads for 1D Cyclic geometric data decomposition by rows.

As mentioned earlier, this access pattern allows not only very good load balancing, but excellent memory performance and cache behavior, since the size of the cache lines divide the size of the rows, there is no false sharing. It can be appreciated that misses remain very stable even when increasing threads. The difference in behavior for some of the numbers of threads compared to the previous implementation, are likely due to the prefetchers rightfully allocating cache lines in the appropriate private caches of the cores due to the predictable sequential access pattern by rows: not only it does not mislead the prefetcher, it is likely benefiting from it. Regarding the very uniform number of misses for each thread, in a similar way to the previous version, this is likely because most misses are compulsory misses and having no false sharing.

4.5. Strong scalability

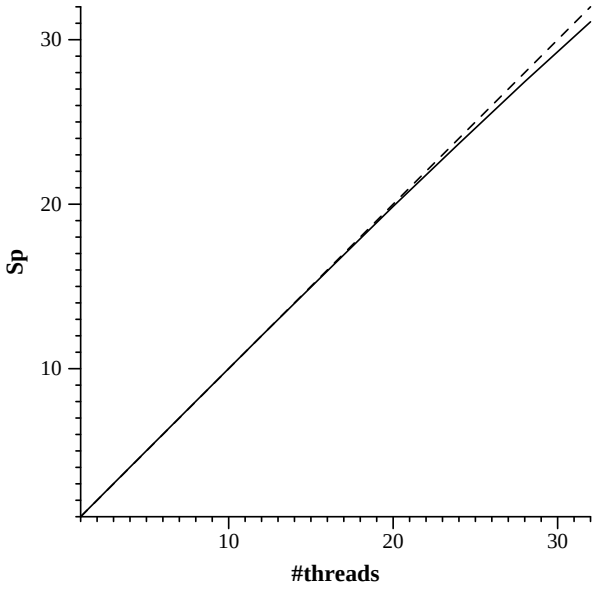

Figure 12: Speed-up with respect to sequential time for 1D Cyclic geometric data decomposition by rows.

The strong scalability plot shows the line consistently extremely close to the ideal as threads keep increasing, just slightly below as it is normal to see, but keeping up with it. This signals a very good strong scalability of this strategy, clearly better than the previous ones.

5. Conclusions

Version	**Number of threads**					
	1	**4**	**8**	**16**	**24**	**32**
1D Block geometric data decomposition by columns	3.227	2.191	1.425	0.7384	0.5528	0.4180
1D Block-cyclic geometric data decomposition by columns	3.231	0.8119	0.4101	0.2146	0.1442	0.1241
1D Cyclic geometric data decomposition by rows	3.208	0.8034	0.4024	0.2022	0.1361	0.1037
Number of threads (L2 cache misses per thread)						
1D Block geometric data decomposition by columns	1612336	474328	277302	160598	120002	61738
1D Block-cyclic geometric data decomposition by columns	1611978	1175316	206223	103392	69188	52000
1D Cyclic geometric data decomposition by rows	1611996	403513	202064	102144	68638	51553

Table 13: Execution time and L2 cache misses per thread for the different geometric data decompositions.

Best parallel implementation

There is little room for discussion, the best parallel implementation is the last one: **1D Cyclic geometric data decomposition by rows**.

Why is it the best overall?

The reason is that this version solves the two main problems presented, to different degrees, by the other two versions:

- the coarse granularity of spatially adjacent data chunks in the first implementation, leading to load imbalance due to the non-uniform workload in the input data;
- further improving that problem as compared to the second implementation, since the rows are one 16th of the size of the column blocks, allowing finer granularity for better workload distribution among threads without increasing cache misses;
- lowering the amount of misses from the other two implementations, likely due to prefetchers correctly allocating spatially close cache lines to the right thread, since the boundary between memory used by different threads is only between the end of a row and the beginning of the next, unlike with the other implementations where, by design, it spans across columns several times.

Overall, presenting the best execution times and best scalability in terms of speedup, due to the better workload distribution that the memory access pattern allows to do without degrading cache performance, and the lowest number of cache misses and better predictability, precisely due to the access pattern used.